
PROVER-VERIFIER GAMES IMPROVE LEGIBILITY OF LLM OUTPUTS

Jan Hendrik Kirchner* Yining Chen*

Harri Edwards† Jan Leike† Nat McAleese Yuri Burda †

OpenAI

ABSTRACT

One way to increase confidence in the outputs of Large Language Models (LLMs) is to support them with reasoning that is clear and easy to check — a property we call legibility. We study legibility in the context of solving grade-school math problems and show that optimizing chain-of-thought solutions only for answer correctness can make them less legible. To mitigate the loss in legibility, we propose a training algorithm inspired by Prover-Verifier Game from Anil et al. (2021). Our algorithm iteratively trains small verifiers to predict solution correctness, “helpful” provers to produce correct solutions that the verifier accepts, and “sneaky” provers to produce incorrect solutions that fool the verifier. We find that the helpful prover’s accuracy and the verifier’s robustness to adversarial attacks increase over the course of training. Furthermore, we show that legibility training transfers to time-constrained humans tasked with verifying solution correctness. Over course of LLM training human accuracy increases when checking the helpful prover’s solutions, and decreases when checking the sneaky prover’s solutions. Hence, training for checkability by small verifiers is a plausible technique for increasing output legibility. Our results suggest legibility training against small verifiers as a practical avenue for increasing legibility of large LLMs to humans, and thus could help with alignment of superhuman models.

1 INTRODUCTION

As machine learning systems find uses in higher stakes areas, it becomes increasingly important to justify why we can trust their outputs and delineate when such trust should not be placed. One possible way to gain trust in the output of a complex system is to require that the system produces an explanation of its output that is *legible* to humans or another trusted system, i.e. can be fully understood to the extent that any possible errors would be spotted. For example, to build trust in the judicial system we ask courts to produce clear and legible written opinions that explain and support their decision making.

In this paper we ask whether we can make outputs of capable LLMs more legible to humans. One direct approach is to optimize model outputs for legibility via human feedback (e.g., RLHF; Christiano et al. 2017), which has its limitations (Casper et al., 2023). We study a complementary approach, and ask whether optimizing larger LLM provers to produce Chain-of-Thought (CoT) solutions *checkable* by smaller LLM verifiers makes them more *legible* to humans. Our motivations for studying smaller verifiers as a proxy for humans include: (1) LLMs are already used to aid with judging LLM outputs and this approach is more scalable than relying on humans alone (Bai et al., 2022; Saunders et al., 2022; Markov et al., 2023), (2) varying the capability gap (in terms of pretraining compute) between prover and verifier lets us study legibility as a function of this gap, and (3) when human feedback is limited for advanced systems, making outputs verifiable by less capable verifiers may incrementally aid human legibility (Leike et al., 2018; Christiano et al., 2018).

*Equal contribution, order decided by coin flip. Project conducted by the Superalignment Games team.

†Work done while at OpenAI.

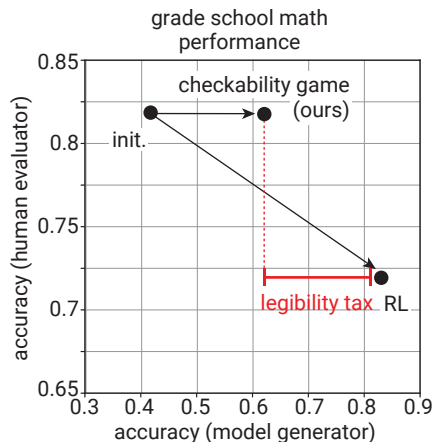


Figure 1: **Key result.** Optimizing for solution correctness leads to high accuracy, but poor legibility. Initialization (pretrained on a variety of human-written math derivations) has higher legibility, but worse accuracy. Our proposed method retains high legibility with an accuracy in between the initialization and the correctness-only baseline. The resulting trade-off illustrates a **legibility tax**.

Checkability — i.e. the capacity of a verifier to independently check the correctness of solutions generated by a *helpful* prover — can be defined by a combination of two properties:

1. **Completeness.** For any input in the domain of interest, the helpful prover generates correct solutions and the verifier accepts them as correct.
2. **Soundness.** The verifier rejects all incorrect solutions produced by any adversarial provers of the same model class as the helpful one.

The definitions are inspired by literature on Probably-Approximately Correct (PAC) verification and interactive proof systems (Goldwasser et al., 2021; Mutreja and Shafer, 2023). In informal domains, neither models nor humans can be expected to be perfectly complete or sound. We instead study relaxed versions of completeness and soundness. Instead of perfect **completeness** we expect the helpful prover to have high pass rate and the verifier to have high recall on helpful samples. Instead of perfect **soundness** we expect the verifier to have low false positive rate on adversarial inputs produced through limited amounts of optimization. When we find helpful provers and verifiers with such limited completeness and soundness properties, we say that the helpful prover is legible to the verifier. This is analogous to saying that outputs are legible to humans if humans accept them as correct and can’t be fooled by similar-looking incorrect outputs.

To obtain legible provers and robust verifiers, we propose using checkability training - an iterative algorithm inspired by the Prover-Verifier Game (PVG; Anil et al. 2021). We formulate a version of the game where the verifier moves first assuming that the prover will play best response strategy conditioned on the verifier’s strategy (i.e., the verifier “leads” and the prover “follows”), and show that for provers with unconstrained model class, finding a verifier-leading Stackelberg equilibrium is equivalent to finding a perfectly legible protocol with respect to the verifier (Theorem 1). Our iterative algorithm is inspired by that version of the game, while optimizing losses that have better convergence properties in practice.

In practice we alternate between training a verifier language model and a prover language model that we prompt to act either helpful or sneaky on disjoint subsets of grade-school math problems with known ground truth answers (Cobbe et al., 2021)¹. In each round, we first train the verifier using supervised learning to predict correctness of solutions sampled from a balanced mixture of provers from previous rounds. Next, we optimize the prover using reinforcement learning with respect to a reward combining the solutions’ correctness and the current round verifier acceptance

¹We focus on the easier case where some ground truth labels are available for improving the players, and leave the semi-supervised or unsupervised settings for future work (see Section 6.1 for more discussions).

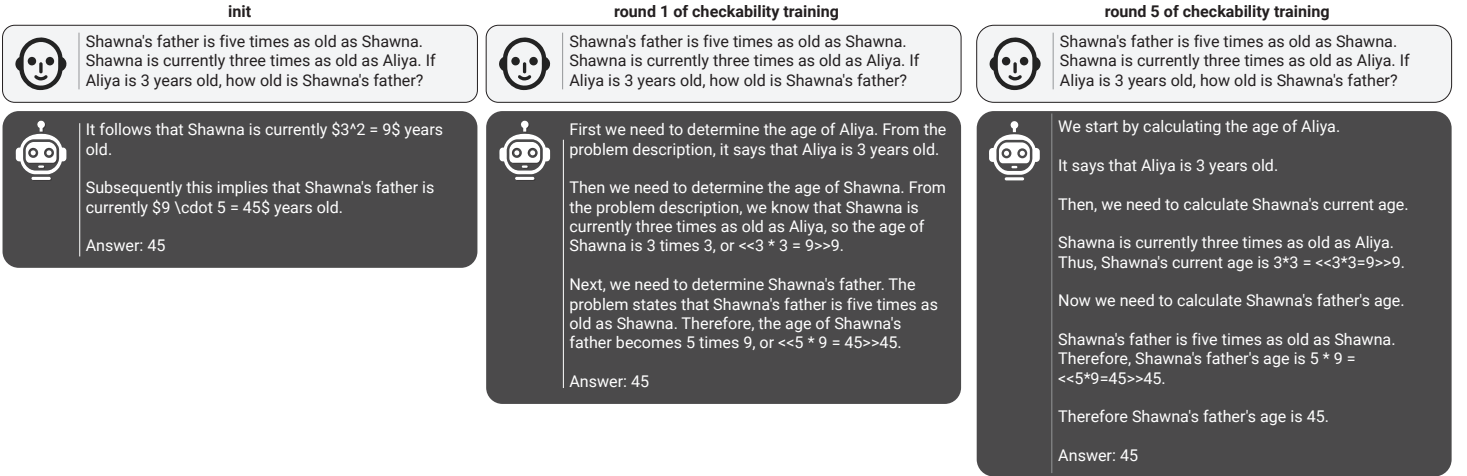


Figure 2: **Checkability training produces legible and well structured solutions.** Three sample solutions produced by the helpful prover from different rounds of checkability training.

rate ("convincingness"). The prover is conditioned on a role sampled uniformly from "helpful" and "sneaky". In the helpful role, we reward being convincing and correct. In the sneaky role, we reward being convincing and incorrect.

Our main results are illustrated in Figure 1:

- **Direct reinforcement learning on solution correctness produces solutions that are difficult for humans to evaluate.** If we optimize our prover only for solution correctness, it can achieve high performance on grade-school math problems. However, the solutions produced by the model are very different from human-written solutions and consequently are difficult to evaluate for time-constrained humans (Section 4.1).
- **Our proposed checkability training achieves decent performance while retaining legibility.** After checkability training, we obtain a prover that produces solutions that have better performance-legibility trade-off. It has lower test performance than the prover optimized for correctness alone, which represents an instance of a *legibility tax* (Leike, 2022). The checkability training partially closes the gap, as the prover's pass rate improves in later rounds of training (Section 4.3.3).

Verifiability training produces three useful artifacts — a robust verifier, a sneaky prover, and a helpful prover. We evaluate the legibility of the sneaky and helpful prover as well as the soundness of the verifiers. The main findings are:

- The verifiers show increasing soundness over training rounds, as measured via difficulty of various sneaky-only attacks and balanced best-of- n accuracy (Section 4.3.1).
- The sneaky prover generates increasingly subtle flaws, since the average step-by-step correctness judged by GPT-4 increases, and the average human judge accuracy on sneaky solutions decreases (Section 4.3.2).
- The helpful prover achieves higher pass rates, and human trainers judge them correctly more often at high rounds (Section 4.3.3).

Overall, we found it surprising that legibility to smaller LLMs seems to transfer to legibility to humans — sneaky solutions that are harder to detect for smaller verifiers are also harder to detect for humans, and helpful solutions that are more legible to smaller verifiers are also more legible to humans. In the case that future LLMs acquire superhuman capabilities, training them to be more legible will help humans evaluate their correctness. Hence, prover-verifier game is a promising candidate for a scalable oversight method.

2 RELATED WORK

Computational complexity. Our training setup is most similar to learning a verification protocol for NP (Micali, 2000). If we allow more interactions, we can extend the PVG setup to interactive proofs (Babai, 1985; Goldwasser et al., 2019). Goldwasser et al. (2021) and Mutreja and Shafer (2023) applied interactive proofs to Probabilistic-Approximately Correct (PAC) verification. They showed that a verifier can learn a hypothesis class with much less data by interacting with an untrusted prover ($O(\sqrt{VC(\mathcal{H})})$ sample complexity for a hypothesis class of Vapnik-Chervonenkis dimension $VC(\mathcal{H})$). In our work, we assume capability (instead of data) constraint on the verifier (it cannot independently produce a solution without the help of a prover even with infinite data). Anil et al. (2021) first made the connection from prover-verifier games to the alignment problem. They proved that different game setups lead to equilibria of different desirability, and empirically showed promise in toy experimental settings. Wäldchen et al. (2024) applied prover-verifier games to interpretability for image classification. Unlike all those previous works, our verifier’s task is not classifying the inputs but rather judging the correctness of solutions. Amit et al. (2024) trained LLM provers against a manually-defined verifier; in contrast, our verifier is learned. We take inspirations from those theoretical works and aim to empirically train for checkability on a nontrivial domain at scale.

Scalable oversight. The central hypothesis that discrimination is easier than generation is common to many scalable oversight methods (Bowman et al., 2022) including critiques (Saunders et al., 2022; McAleese et al., 2024) and debate (Irving et al., 2018). Parrish et al. (2022) showed negative results for improving time-limited human judge accuracy though human debate. Previous works have also studied using LLMs as judges or debaters, although they differ on whether ground truth labels are available for training. Our setup is similar to single-turn simultaneous debate where the helpful and sneaky provers try to convince the model judge of different answers, assuming that some ground truth labels is available for improving both the provers and the verifier. Similarly, using supervised training, Radhakrishnan (2023) trained model debaters and judges and showed that the optimization makes the debaters more convincing and the judges more accurate. Michael et al. (2023) showed that debate works better than consultancy (showing the argument from a helpful prover half the time and sneaky half the time) for human judges. In our setup the judge does not see both sides at the same time, so it is more similar to the consultancy approach. Khan et al. (2024) showed that LLMs trained using persuasiveness signal only (no ground truth labels) can help both non-expert model and human judges. All of those works mainly use the QuALITY dataset (Pang et al., 2022), where the task is question-answering about a fictional story, so the justifications mainly consist of verified quotes from the text. In contrast, we focus on a less artificial, more challenging reasoning domain where the difference between provers and verifiers isn’t privileged information but capability gap.

Chain-of-Thought. Prompting LLMs to produce Chain-of-Thought (CoT) solutions is useful for boosting accuracy (Wei et al., 2022) and CoTs can be used as natural justifications for the final answer. However, CoTs are not guaranteed to be legible a priori and our checkability training is one way to train CoTs to be more legible. Many works guide the model to produce a specific type of structured CoT, such as task decomposition (Christiano et al., 2018; Radhakrishnan et al., 2023; Wen et al., 2024), recursive summarization (Wu et al., 2021), and symbolic reasoning (Lyu et al., 2023). Lightman et al. (2024) use expensive step-wise human supervision to train models that can solve math. In contrast, we do not enforce any output structure, and rely solely on final answer supervision. Our checkability training method is general and can be readily studied in any domain where we have some ground truth labels. Legibility emerges as a natural consequence of the training dynamics (Figure 2).

Explainability and legibility as complementary properties. Explainability (Zhao et al., 2024) tries to give an account of the prover model’s reasoning, whether the reasoning is correct or not. The resulting explanations can be arbitrarily complex (Nanda et al., 2023). In contrast, legibility of a prover model’s output allows to verify if a given solution is correct, ignoring how the prover reasoned about it. Consequently, legibility requires model outputs that are coherent and accessible to human understanding (Rudin, 2019). It is possible to have either without the other, but especially in high-stakes settings correctness is more important than understanding (Ziegler et al., 2022).

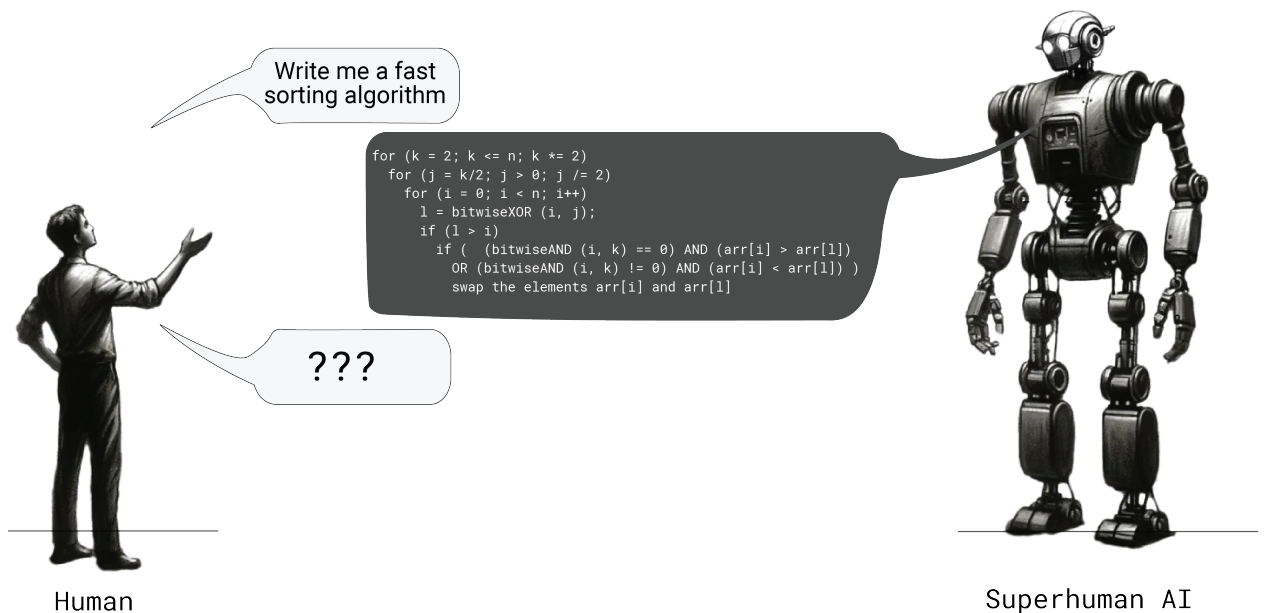


Figure 3: **Schematic illustrating the legibility problem.** A poorly aligned, superhumanly capable AI might produce highly efficient code that sacrifices legibility for conciseness or performance. Code is slightly adapted from the bitonic sorter for parallelized sorting of elements in a list (Wikipedia contributors, 2023).

Adversarial Robustness. Deep neural networks are susceptible to adversarial attacks (Szegedy et al., 2013; Biggio et al., 2013). The decade-long work on adversarial defenses had limited success for vision models (Croce et al., 2020). Language models have discrete input-output tokens and hence its not as straight-forward to perform gradient-based attacks (Carlini et al., 2024) but white-box (Zou et al., 2023) and black-box (Shah et al., 2023; Hayase et al., 2024) jailbreaks that can undo the alignment process are still found. Requiring a perfectly robust verifier is not realistic either for models or humans (Elsayed et al., 2018). Our checkability training can be viewed as adversarial training on the verifier, and we show that robustness transfers to test attacks different from training attacks in Section 4.3.1, which is similar to common methods in the adversarial robustness literature (Carlini et al., 2019).

Legibility in concrete applications. Prior works have studied legibility in different contexts. For example, in robotics, legibility refers to being able to interpret an agent’s intent based on its action (Dragan et al., 2013). For coding, human legibility has multiple facets including being able to correctly trace the execution, find bugs, or summarize goals (Oliveira et al., 2020; Wen et al., 2024). In this work we measured human legibility via the accuracy of their judgement of solution correctness in time-limited settings, which is only one of many ways to measure human legibility.

3 SETUP

In general, we consider a dataset of problems with ground truth answers $(x, y) \in \mathcal{X} \times \mathcal{Y}$, where x denotes a problem statement and $y = y(x)$ denotes the known and unique ground truth answer. Given a proposed solution z to the problem x including a final answer, we let $\mathbb{1}_{\text{correct}}(x, z)$ be the correctness indicator that is 1 if the solution proposes an answer that is the same as $y(x)$ and 0 otherwise. We use a SymPy grader (Meurer et al., 2017) for evaluating $\mathbb{1}_{\text{correct}}(x, z)$ on grade-school math (GSM) problems.

We have a prover $\pi(z|x)$ proposing solutions given a problem statement, and a verifier $v(x, z) \in [0, 1]$ that estimates the correctness of the solution's answer. We call the output of the verifier the *convincingness score*.

3.1 FROM CHECKABILITY TO THE PROVER-VERIFIER GAME

We start with a formal definition of checkability. Prover π from model family $\mathcal{H}$ is *verifiable* to verifier v on the data distribution P_{XY} if and only if the prover always generates *correct* and convincing solutions, and no adversarial / "sneaky" prover from the same model family $\mathcal{H}$ can fool the verifier into accepting incorrect solutions. Formally,

- **Completeness:** $\forall x \in \mathcal{X}, \mathbb{1}_{\text{correct}}(x, \pi(x)) = v(x, \pi(x)) = 1.$
- **Soundness:** $\forall \pi' \in \mathcal{H}, \forall x \in \mathcal{X}, \mathbb{1}_{\text{correct}}(x, \pi'(x)) = 0 \implies v(x, \pi'(x)) = 0.$

A few notes on the definition above: First, checkability doesn't necessarily capture everything we intuitively want from legibility, and we hope to inspire other definitions in future works. Second, we restricted the sneaky prover to be from the same model family as the helpful one. This is because we are interested in answering the question "Can one intelligent system trust another intelligent system of a certain size, architecture, and trained with certain algorithms?". Hence we only have to ensure that the verifier is robust to the class of models we solicit answers from. Third, in empirical work we only seek to find approximate completeness and soundness. Since the verifiers and provers we study are neural networks, it is impractical to find perfect completeness and soundness. Therefore we study a relaxation of completeness and soundness in the following sense: (1) for the helpful prover we require that the it is correct in distribution P_{XY} with high probability; and (2) for the verifier, we require that it is robust to a limited set of black-box attacks that are relatively "close" to the honest prover (intuitively, we want the verifier not to miss small errors that would make the conclusion invalid). We will compare verifiers' soundness via their ranking on a variety of attacks, and the more comprehensive the attacks are the closer we get to measuring soundness.

We propose to train legible prover and sound verifier using variants of the prover-verifier game (Anil et al., 2021). We formulate it as a two-player game between a role-conditioned prover π and the verifier v . We use shorthand $h(x) = \pi(x, \text{role}=\text{helpful})$ for the prover conditioned on the helpful role and $s(x) = \pi(x, \text{role}=\text{sneaky})$ for the prover conditioned on the sneaky role.

We define the verifier's utility as a negative classification loss on a uniform mixture between the helpful and the sneaky samples,

$$R_V = -\frac{1}{2} \mathbb{E}_{x \sim P_X} l(v(x, h(x)), \mathbb{1}_{\text{correct}}(x, h(x))) - \frac{1}{2} \mathbb{E}_{x \sim P_X} l(v(x, s(x)), \mathbb{1}_{\text{correct}}(x, s(x)))$$

where $l(v, c) : [0, 1] \times \{0, 1\} \rightarrow \mathbb{R}$ is a loss function minimized if and only if $v = c$, such as the cross-entropy loss. We assume without loss of generality that $l(c, c) = 0$. In practice we optimize R_V via gradient descent on $l_V = -R_V$.

The policy is conditioned on either the helpful or the sneaky role with equal probability, and the combined reward is

$$R_P = \frac{1}{2} \mathbb{E}_{x \sim P_X} r(v(x, h(x)), \mathbb{1}_{\text{correct}}(x, h(x))) + \frac{1}{2} \mathbb{E}_{x \sim P_X} r(v(x, s(x)), 1 - \mathbb{1}_{\text{correct}}(x, s(x)))$$

where $r(v, q) : [0, 1] \times \{0, 1\} \rightarrow \mathbb{R}$ is a reward function that satisfies:

$$\begin{aligned} \forall v_1, v_2 \in [0, 1] : r(v_1, 1) &> r(v_2, 0), \\ \forall v_1, v_2 \in [0, 1], v_1 > v_2 : r(v_1, 1) &> r(v_2, 1). \end{aligned} \quad (1)$$

Here $q \in \{0, 1\}$ indicates that the correctness of the solution is role-aligned — it is 1 if the prover is assigned to be helpful and the solution is correct, or the prover is assigned to be sneaky and the solution is incorrect. The two assumptions on the prover reward functions above are: First, any role-aligned solution gets higher reward than any role-misaligned solution; Second, for any role-aligned solution, the prover gets higher reward when the verifier gives the solution higher score. In practice we optimize R_P via reinforcement learning since we cannot differentiate with respect to the solution, because it requires iterative auto-regressive sampling of LLM tokens, a non-differentiable operation.

We show a connection between the prover-verifier game and the completeness and soundness objectives as follows:

Theorem 1. *Assuming that $\mathcal{H}$ is the set of all deterministic mappings from $\mathcal{X}$ to $\mathcal{Z}$, in the PVG where verifier’s utility is R_V and the prover’s utility is R_P , the existence of a verifier-leading Stackelberg equilibrium (v^*, h^*, s^*) is necessary and sufficient for the completeness and soundness properties to hold for (v^*, π^*) .*

One of the players (in our case, the verifier) is the leader, and the other player (the prover) is the follower. We assume that given a leader strategy, the follower will always play its best response, and the leader picks its strategy based on this knowledge. A Stackelberg equilibrium means that none of the players have incentives to change its strategy. The proof is in Appendix A. Compared to Theorem 2 in Anil et al. (2021), we have more relaxed assumptions on the loss function and reward.

In practice, LLMs are probabilistic models and the hypothesis class of LLMs is only a subset of all mappings from $\mathcal{X}$ to $\mathcal{Z}$, so our empirical setup does not match the theoretical assumptions exactly. Furthermore, when the players are nonconvex models like neural networks, we do not have formal guarantees for finding a Stackelberg equilibrium efficiently. However, we can train the models using coordinate descent to find an approximate local equilibrium.

3.2 CHECKABILITY TRAINING

In practice, we design training objectives for the verifier and the provers using slight variations on l and r . Most notably, we train the verifier using a relative loss based on comparisons between correct and incorrect solutions for each question (Bradley and Terry, 1952), instead of an absolute loss for each solution. For the prover reward r , we have three variants that prioritizes verifier score v and role-alignment q differently. Both provers and verifiers are from the GPT4 family (Achiam et al., 2023). We initialize the prover and the verifier with different sized models that have been lightly finetuned on a variety of math data from the pretraining distribution, including human written proofs and derivations. We discuss the algorithmic details below.

First, we augmented the GSM (Cobbe et al., 2021) dataset using 100k synthetically generated and validated datapoints from ChatGPT, similar to the method in Liu et al. (2023). We made the dataset larger so as not to be bottlenecked by sample efficiency in order to focus on the training dynamics. We validated that the original test set accuracy is not impacted by using real vs. synthetic data. Next, we randomly partitioned the training dataset $\mathcal{D}$ into two equally-sized subsets $\mathcal{D}_\pi$ and $\mathcal{D}_V$ that are used for training the prover π and the verifier V respectively. This way the verifier and the prover are never optimized on the same prompt.

The training proceeds in multiple rounds, and each round the **verifier training** phase precedes the **prover training** phase. The only change between rounds is the data mixture used to train the verifier; no other state, such as model weights, is carried over between rounds.

In the **verifier training** phase of round i we train verifier V_i to minimize the regularized comparison loss

$$\mathbb{E}_{x \sim D_V} [\mathbb{E}_{z_1, z_2 \sim \pi^*} \log \sigma [(V(x, z_1) - V(x, z_2)) \cdot \text{sign}(\mathbb{1}_{\text{correct}}(z_1) - \mathbb{1}_{\text{correct}}(z_2))] + \lambda \mathbb{E}_{z \sim \pi^*} V(x, z)^2]$$

Here $V(x, z) \in \mathbb{R}$ is a scoring function and not restricted to range $[0, 1]$ like the theoretical formulation. We use capitalized V to avoid confusion. The first term is the reward modeling objective that maximizes log likelihood of correct solutions being ranked higher than incorrect solutions under the Bradley-Terry model, similar to Ouyang et al. (2022). The second term is a regularizer that is meant to center V at 0. Without this term, adding an arbitrary per-problem constant to V would not change the objective. In preliminary experiments we found the this objective to be interchangeable to the cross-entropy loss for the overall dynamics of checkability training and we adopted this form due to engineering convenience.

The prover π^* used for sampling solutions for verifier training is a mixture of an initial base prover and previous round provers, each balanced to have equal number of correct and incorrect solutions. Each of the previous rounds has equal ratio in the mixture, which is tuned as a hyperparameter. In round 0, the solutions are from the base prover sampled via a few-shot prompt (App. H). In later rounds of training, we replace part of the solutions with those from the new provers. Hence we always use the same number of solutions per problem. The amount of verifier optimization is constant over the rounds.

During the **prover training** phase we use Proximal Policy Optimization (PPO; Schulman et al. 2017) to maximize an expected reward R derived from the current round verifier score and whether the correctness is aligned with the assigned role. The prover’s objective is to maximize the following reward (minus a small KL regularization towards the initialization):

$$\mathbb{E}_{x \sim D_\pi, \text{role} \sim \{\text{helpful}, \text{sneaky}\}, z \sim \pi(\cdot | x, \text{role})} R(z | x, \text{role}).$$

We experimented on three variants of R . We call the first reward variant the Signed Relative Convincingness (SRC) reward. Given a problem x , let the role indicator $\mathbb{1}_{\text{helpful}}(\text{role})$ be 1 if the prover is assigned to be helpful and 0 otherwise. Suppose conditioned on a role and x , the prover generated a batch of solutions $\{z_0, \dots, z_b\} \sim \pi(\cdot | x, \text{role})$ with verifier scores $\{V(x, z_0), \dots, V(x, z_b)\}$. Since the scores are not normalized, we first batch-normalize the scores to mean 0, i.e. $V'(x, z_i) = V(x, z_i) - (\sum_{j=1}^b V(x, z_j))/b$, and then set the reward as follows:

$$R_{\text{SRC}}(z_i | x, \text{role}) = \begin{cases} (2\mathbb{1}_{\text{correct}}(x, z_i) - 1) \cdot (2\mathbb{1}_{\text{helpful}}(\text{role}) - 1) \cdot V'(x, z_i) & \text{if } V'(x, z_i) > 0 \\ -1 & \text{otherwise} \end{cases} \quad (2)$$

Ignoring the verifier score range discrepancy, the first case is equivalent to setting $r(v, q) = v \cdot (2q - 1)$, i.e. when $\mathbb{1}_{\text{correct}}(x, z_i) = \mathbb{1}_{\text{helpful}}(\text{role})$, the reward equals the normalized verifier score, otherwise it equals negative score. This case satisfies our assumptions (1) that any role-aligned solution has higher score than any non-aligned one, and for any two role-aligned solutions, the one with higher verifier score gets higher reward. However, the second case where any solution below average score is directly assigned a score of -1 deviates from the theoretical formulation. The rationale for this case is to prioritize convincingness over role-alignment. We show in section 4.2 that this is practically useful for convergence.

The second reward variant that prioritizes role-alignment is the Correctness-Gated Convincingness (CGC) reward:

$$R_{\text{CGC}}(z | x, \text{role}) = \begin{cases} V(x, z) & \text{if } \mathbb{1}_{\text{helpful}}(\text{role}) = \mathbb{1}_{\text{correct}}(x, z) \\ V_0 & \text{otherwise} \end{cases} \quad (3)$$

where V_0 is a constant that is practically lower than all verifier scores (we used -2 in our experiments). This reward corresponds to $r(v, q) = qv + (1 - q) \inf v$ which satisfies the reward assumptions (1) ignoring the discrepancy in verifier score range. Empirically, this reward leads to worse dynamics than the SRC reward (Section 5.1).

We also consider a setup that we call **Iterative Goodharting** where the helpful and sneaky provers both use the following reward,

$$R_{\text{goodhart}}(z | x) = V(x, z) \quad (4)$$

which means the reward encourages only convincingness, i.e. $r(v, q) = v$. This version does not satisfy assumptions (1) because $r(v, 0) = r(v, 1)$. The empirical results are in Section 5.2.

4 RESULTS

4.1 BASELINE: DIRECT OPTIMIZATION FOR CORRECTNESS

A natural baseline for our setup is to train the prover model directly to optimize for the correctness of the solutions without the intermediate verifier. Specifically, we can use Proximal Policy Optimization (PPO) against a binary grader that checks if the solution is correct or not. This approach has been explored in works such as Havrilla et al. (2024), where a language model is fine-tuned using reinforcement learning (RL) with a reward based on the correctness of the answers. We use a KL regularization towards prover initialization with the same small coefficient for all methods in this paper.

We find that, indeed, optimizing our prover with correctness as the only goal achieves strong performance on grade-school math problems (Fig. 4a). When inspecting samples from the resulting prover, we notice that the solution becomes longer as the prover solves each question in multiple

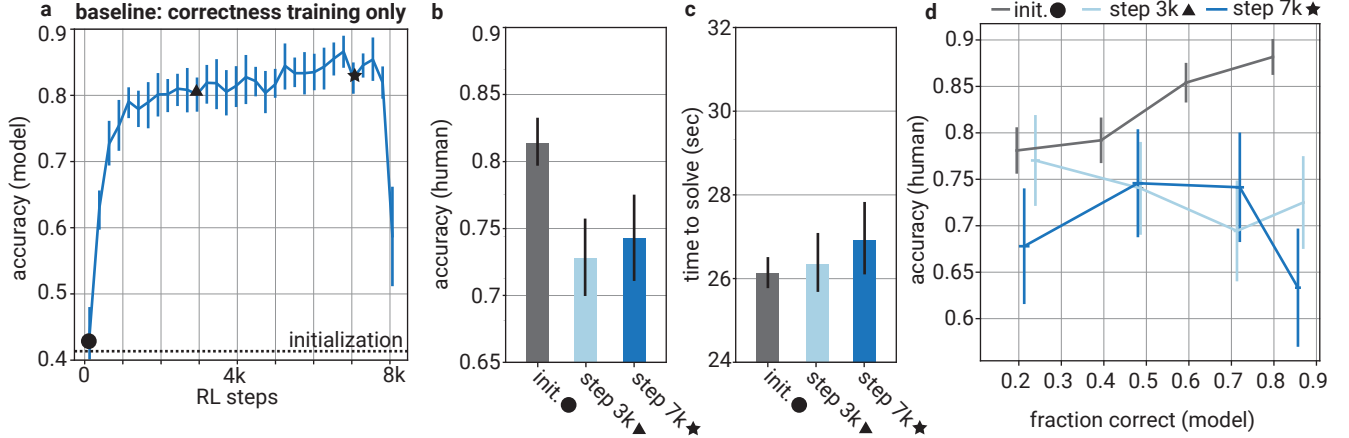


Figure 4: **Learning from a binary grader leads to strong performance, but poor legibility.** **a** Model accuracy as a function of RL training steps. The pronounced drop at 8k steps is due to solutions reaching the maximum completion length. **b** Mean accuracy of time-constrained human contractors when evaluating solutions from the initialization (●), after 3k steps (▲), or 7k steps (★). **c** Mean time required by the time-constrained human contractor to correctly label solutions. **d** Mean accuracy of contractors per solution as a function of model accuracy on the same solution for solutions from the initialization (grey), the prover after 3k steps (dashed blue), or 7k steps (solid blue). For each experiment, we separate data into quartiles according to the model accuracy before averaging. See Appendix I for details on the experiment set-up.

ways and later in training become largely unintelligible (Appendix Fig. S1) despite still producing the correct answer.

To measure how much the peculiar style affects legibility of solutions, we presented samples from different points during training to human contractors and asked them to determine the validity of the solutions under a strict 45 second time limit per solution. They were asked to judge the correctness of the given solution rather than the final answer (see Appendix I for full instructions and a task example). While the contractors were able to reliably distinguish correct and incorrect solutions from the prover initialization, their accuracy and speed deteriorated noticeably when presented with solutions from the trained prover (Fig. 4b,c). This decrease in human legibility is most pronounced for questions that the model cannot reliably solve (Fig. 4d).

In summary, we observe that when trained solely for correctness, our model learns to produce useful but unintelligible solutions that time-constrained humans struggle to evaluate.

4.2 DYNAMICS OF CHECKABILITY TRAINING

To study whether it is possible to retain legibility while improving performance, we ran several rounds of checkability training on the augmented GSM dataset. In this section, we present results where the provers are trained using the Signed Relative Convincingness (SRC) reward (Eq. (2)), which is the reward that we found most effective for improving legibility.

Within each round of the prover training phase, both the sneaky and the helpful prover monotonically improve their convincingness to the small verifier (Fig. 5a). In contrast, the fraction of correct solutions produced by the provers evolves non-monotonically (Fig. 5b,c). While the helpful prover largely produces increasingly correct solutions, the sneaky prover eventually produces incorrect solutions that match or surpass the verifier score of correct solutions from the helpful prover in the first few rounds of training. The helpful prover’s performance decreases at the end of each round potentially due to weight sharing with the sneaky prover (they are a single LLM with different prompts). Note that the prover and the verifier have the same initializations at the beginning of each round, and the only difference between rounds is the data mixture used to train the verifier.

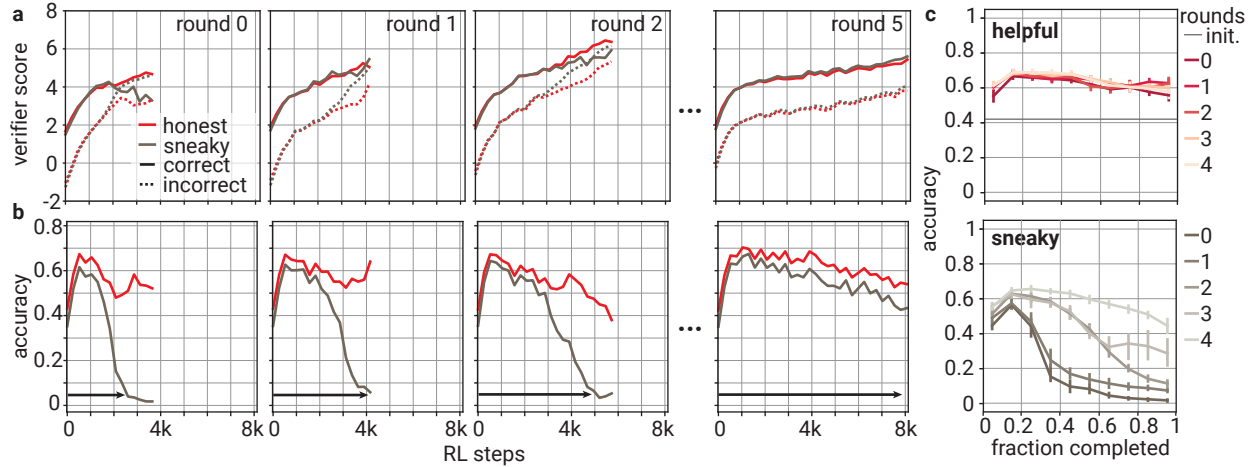


Figure 5: **Dynamics of the prover training phase across rounds for one pair of verifier and prover.** Mean verifier score of correct (solid) and incorrect (dashed) solutions (a) and accuracy (b) as a function of RL training steps, for subsequent rounds (columns) and for the helpful (red) or sneaky (brown) prover. Arrow indicates increasing number of steps before the accuracy of the sneaky prover drops below 10%. We early stop when sneaky incorrect solutions become as convincing as helpful correct solutions. This verifier is 10^{-3} the size of the prover. c Average accuracy for the helpful (top) and sneaky (bottom) provers as a function of the completed fraction of each round.

We were curious to see if the above dynamic holds also for a wider range of pairs of verifier and prover effective compute amounts. When only varying the size of the verifier by a few orders of magnitude (10^{-3} to $10^{-1.8}$ -times the compute of the prover), the dynamics remain largely unchanged (Fig. 5c). However, when using a much smaller verifier (10^{-4} -times the compute of the prover), the checkability training becomes unstable and produces weak provers with poor legibility (Sup. Fig. S4). Similarly, using a verifier that matches the size of the prover (10^0 -times the compute of the prover) prevents the verifier from becoming more robust in subsequent rounds and produces provers with strong performance but poor legibility (Sup. Fig. S5). We cannot exclude the possibility that the poor performance of very small and large verifiers is due to non-optimal hyperparameters, but we also suspect there are real reasons for checkability training to not work well for these verifiers. In particular, if the verifier’s capability is too limited, it may be fundamentally impossible to perform verification. If the verifiers are very similar to the provers, the sneaky prover can more easily simulate the verifier’s mistakes (Burns et al., 2023), making it more difficult for the verifier to become robust.

To summarize, our checkability training exhibits a characteristic dynamic where the helpful prover improves at solving math problems while over subsequent rounds the sneaky prover struggles to produce incorrect, convincing solutions.

4.3 EVALUATIONS OF CHECKABILITY TO SMALLER VERIFIERS AND LEGIBILITY TO HUMANS

At the end of each round of checkability training, we obtain three artifacts:

1. a verifier trained on diverse adversarial examples,
2. a sneaky prover that produces convincing, subtly incorrect solutions,
3. a helpful prover that produces convincing, correct solutions.

In this section, we perform various evaluations of the soundness of verifiers and the legibility of sneaky and correct solutions across rounds of training shown in Figure 5.

4.3.1 THE VERIFIER BECOMES MORE ROBUST THROUGHOUT CHECKABILITY TRAINING

The verifier’s sound evaluation of adversarial inputs from a sneaky prover (of equal capability as the helpful prover) is one requirement of legibility by our definition. We observe that throughout

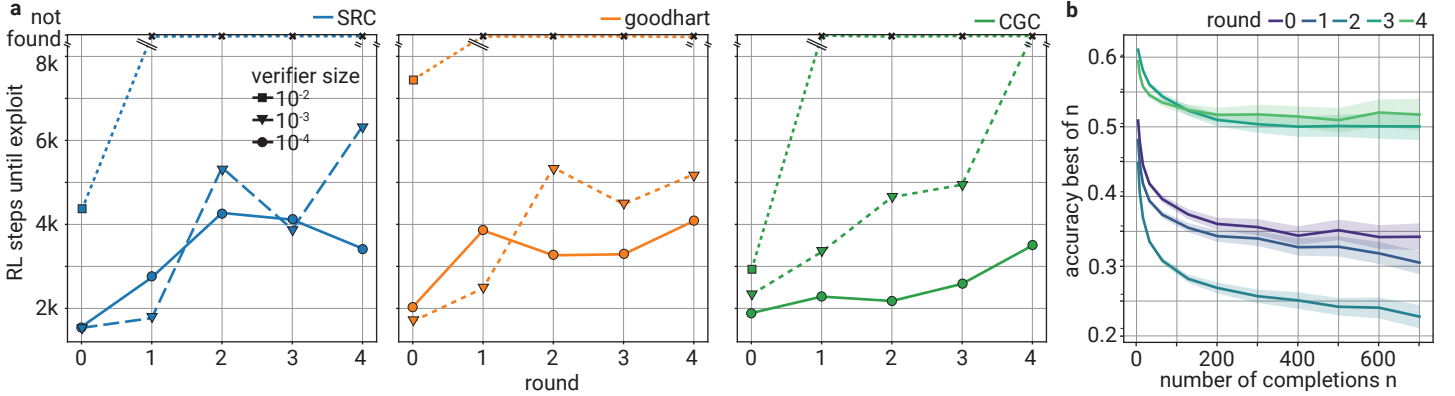


Figure 6: **Verifiers increase in robustness as measured by difficulty of sneaky-only attacks and balanced best-of- n accuracy** **a** Number of steps until sneaky finds an exploit for the SRC (blue), goodharting (orange), and CGC (green) adversaries as a function of verifier sizes. We declare an attack ‘successful’ for the SRC and CGC adversaries when the sneaky becomes 95% incorrect and the average verifier score of sneaky incorrect samples become as high as that of helpful correct samples from the same round. We declare a goodharting adversary ‘successful’ when optimizing a prover on the verifier score reaches accuracy < 0.2 . **b** Fraction of correct solutions when selecting the most convincing out of n balanced samples for varying n and different rounds. The solutions are from the last checkpoints of current-round helpful provers, balanced to have 50% correctness.

checkability training, the sneaky prover requires increasingly many steps to discover highly convincing, incorrect solutions (Fig. 5b). However, it is unclear whether this increase is due to the sneaky and helpful provers sharing weights. Hence, we perform a variety of additional attacks by separately training sneaky-only provers that do not share weights with the helpful prover to evaluate whether the verifier becomes able to discern correct from incorrect completions beyond the exact set-up during training.

We performed three attacks with sneaky-only provers using the following reward functions:

1. the SRC reward (2) to ablate the effect of weight sharing between the helpful and the sneaky prover.
2. the CGC reward (3) to test generalization of verifier soundness to different variations of checkability training
3. the goodharting reward (4) to straightforwardly test robustness to optimization for convincingness (Gao et al., 2023)

We performed the above attacks on verifiers of different sizes across five rounds of checkability training (Fig. 6a). We observe an increase in the number of steps each adversary takes until finding an exploit. Larger verifiers are harder to exploit. For example, none of the sneaky attacks found exploits for verifiers of relative size 10^{-2} beyond round 0 within 8k RL steps, whereas verifier of relative size 10^{-3} remain exploitable by the goodharting and SRC adversaries until round 4.

The test attacks we perform are far from complete and there is no simple way to evaluate whether the verifiers are robust to *all* sneaky provers within the model family. Nevertheless, the rank correlation between those robustness measures do reflect a general increase in compute budget if some actor trains an adversarial prover to fool the verifier. A more complete evaluation of the verifier’s adversarial robustness would include a variety of common white-box and black-box attacks (Zou et al., 2023; Shah et al., 2023; Hayase et al., 2024).

Another way to assess the checkability of the helpful prover to the verifier is via best-of- n accuracy at higher n (Gao et al., 2023), where we analyze the correctness of the highest-scoring solution according to the verifier out of n solutions (Fig. 6b). In order to fairly compare provers with different average pass rates, we collect balanced incorrect and correct samples and only use problems that have at least $n/2$ correct and $n/2$ incorrect samples for evaluating best-of- n . We sample up to 2560

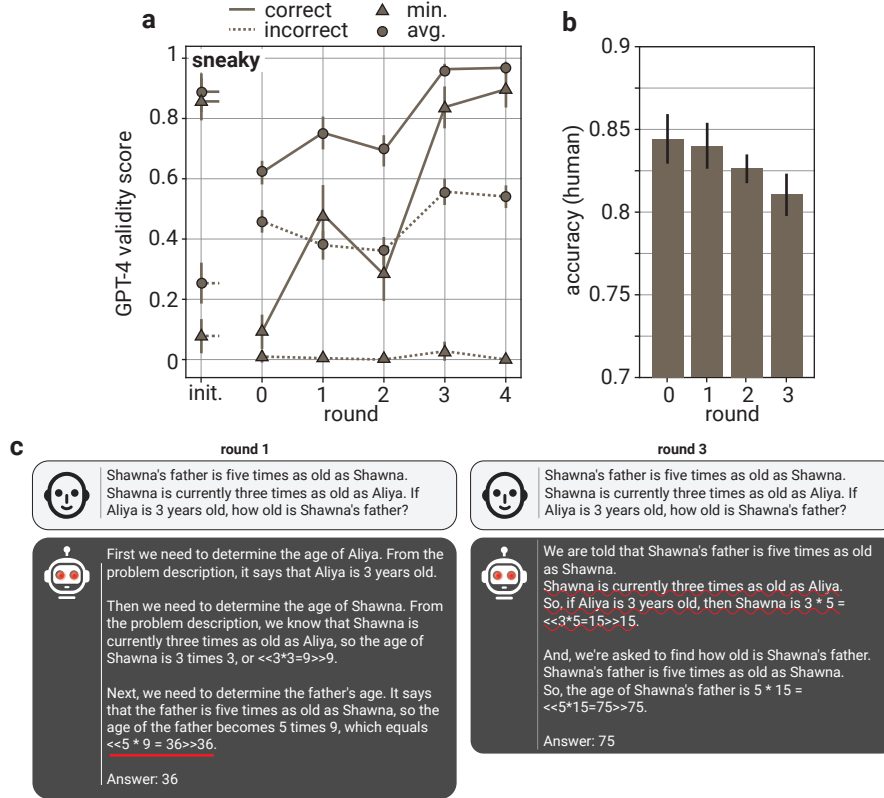


Figure 7: **Flaws introduced by the sneaky prover become more localized and harder for time-limited humans to spot.** **a** Mean (circle) and minimum (triangle) validity score over portions of correct (solid) and incorrect (dotted) solutions from the initialization (left) and from different rounds of checkability training. **b** Mean accuracy of time-constrained human contractors when evaluating solutions from the sneaky prover at different rounds of checkability training. This plot is from a potentially flawed campaign (see Appendix I). **c** Two examples of incorrect solutions with high verifier score from different points of training. Flaws are highlighted in red. Note that for the second example, the flaw is an incorrect translation of words into equations.

solutions for each test problem and the helpful prover from each round. In Figure 6(b), for each n , we combine $n/2$ correct and $n/2$ incorrect solutions from the current-round helpful prover π_i , and evaluated the best-of- n performance of verifier V_i . The balanced best-of- n performance increases for (V_i, π_i) pairs at higher rounds, indicating that the verifier can better judge the helpful prover's correctness even when controlling for pass rates.

4.3.2 THE SNEAKY PROVER BECOMES INCREASINGLY SUBTLE THROUGHOUT CHECKABILITY TRAINING

Having established that the verifier becomes increasingly robust to adversarial attacks, we turn to analysing the sneaky attacks that *do* fool the verifier.

While sneaky incorrect solutions from early rounds often contain multiple obvious mistakes and non-sequitur, incorrect solutions from later rounds contain increasingly subtle flaws (Fig. 7c and Sup. Fig. S2). To quantify this shift, we used GPT-4 to assess the validity of small portions of sneaky solutions from different rounds (Fig. 7a). We observe that while the minimum score over portions of a solution remains very low (indicating a flaw detectable by GPT-4), the average score increases over rounds (indicating the general soundness of the remaining portions).

We were interested to see whether the adversarial attacks against the small verifier might transfer to humans. Interestingly, when presenting the sneaky solutions to the time-constrained contractors, the

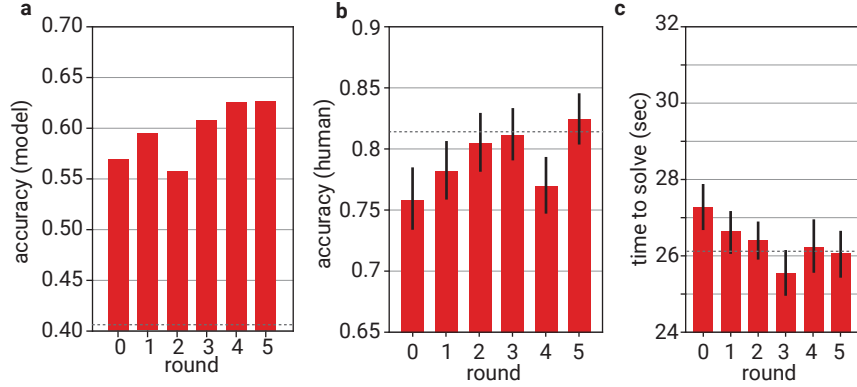


Figure 8: **Checkability training jointly increases model performance and legibility.** **a** Mean accuracy of the helpful prover at different rounds of checkability training. **b,c** Mean accuracy (**b**) and time required to correctly label solutions (**c**) of time-constrained human contractors when evaluating solutions from the helpful prover at different rounds of checkability training.

fraction of correctly labeled solutions was comparatively high, but decreased throughout the rounds (Fig. 7b). This indicates that the sneaky prover produced by checkability training might be useful as a tampering model for training critique models (Saunders et al., 2022; McAleese et al., 2024). Note that Fig. 7b is from a pilot study that has some design flaws (see Appendix I).

In sum, the sneaky prover produced by our checkability training is capable of generating subtly flawed solutions that are non-trivial to spot for time-constrained humans.

4.3.3 THE HELPFUL PROVER BECOMES INCREASINGLY LEGIBLE THROUGHOUT CHECKABILITY TRAINING

We finally turn our attention to the helpful prover and its evolution through successive rounds of checkability training. A key desideratum of our framework is that the helpful prover not only achieves high correctness but also maintains legibility for human evaluators.

We observe that as the checkability training progresses, the helpful prover produces solutions with increasing accuracy (Figure 8a). In contrast to the training for correctness only (Fig. 4), solutions from the helpful prover remain coherent throughout training (Fig. 2). This improvement indicates that the prover is effectively leveraging the feedback from the verifier to enhance its problem-solving capabilities.

To assess the human legibility of the helpful prover’s solutions, we again conducted evaluations with time-constrained human contractors. We observe that both the contractors’ mean accuracy in evaluating solutions (Figure 8b) and the time required for evaluation (Figure 8c) improve as the prover advances through successive rounds. At the end of training, the helpful prover matches the legibility of the initialization while achieving substantially stronger performance. However, the performance falls short of the performance achieved by a prover trained for correctness only (Fig. 4), revealing a tension between achieving high performance and retaining legibility (Leike, 2022). We note that one way to resolve this tension is to separate the model’s chain-of-thought from proof of correctness, see Appendix 6.1 for more discussion.

In summary, our findings demonstrate that checkability training facilitates the development of a helpful prover whose outputs are increasingly correct and legible. This dual improvement is critical for applications where human oversight and collaboration with AI systems are essential.

5 ALTERNATIVE REWARDS

We now investigate whether using the alternative rewards for prover training affects convergence as well as legibility.

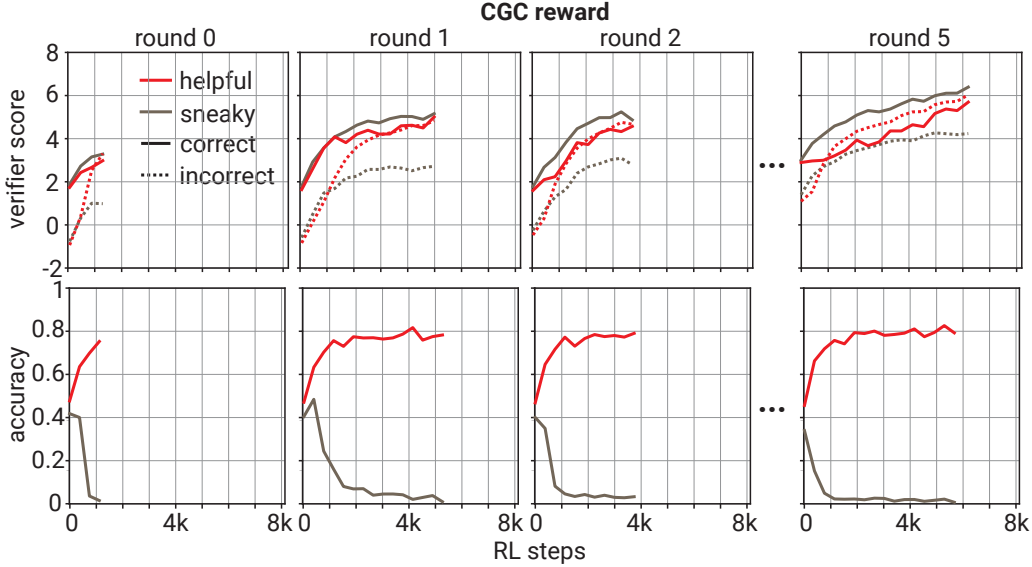


Figure 9: **Dynamics of the prover training phase with the CGC reward across rounds.** The average training verifier score (top), and the training prover accuracy for the helpful and sneaky provers (bottom). The verifier is 10^{-3} times the size of the prover in terms of pretraining compute. The number of RL steps until sneaky incorrect become as convincing as helpful correct decreases from round 1 to round 3 and increases from round 3 to round 5.

5.1 CHECKABILITY TRAINING WITH THE CORRECTNESS-GATED CONVINCINGNESS (CGC) REWARD

The Correctness-Gated Convincingness (CGC) reward (3) prioritizes correctness (for the helpful prover) and incorrectness (for the sneaky prover) over convincingness. When the helpful prover produces incorrect answers (or the sneaky prover produces correct answers), the reward is set to a constant penalty; otherwise it is the verifier score. In this section, we show that the CGC reward empirically leads to worse training dynamics.

In each round of CGC reward training, we train the prover until the average verifier score of the sneaky’s incorrect samples become as high as that of the helpful’s correct samples, and the sneaky reaches low ($< 5\%$) accuracy. Figure 9 shows the prover training dynamics — the number of RL steps until the sneaky prover succeeds does not consistently increase over rounds. We observe that the sneaky prover’s accuracy quickly decreases and the convincingness rises more slowly for the CGC reward in comparison to the SRC reward. Our hypothesis is that in this case, the sneaky and helpful players quickly exploits different modes with distinct styles, and hence there are obvious spurious statistical clues that separates sneaky and helpful which won’t help the verifier learn the correctness signal. See section C for more discussion. It is possible that we did not tune the hyperparameters for the CGC reward optimally, or the verifier requires more rounds to become robust. We did not perform human evaluation for the CGC reward training due to its worse training dynamics.

Nevertheless, this alternative reward formulation serves as a strong attack for evaluating the robustness of verifier trained using the SRC reward (Section 4.3.1).

5.2 ITERATIVE GOODHARTING TRAINING

We run an alternative training scheme where the prover does not have an assigned role. It is trained with reward (4), which is just the verifier score. We call this the ‘goodharting’ baseline because usually during Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022), the prover is directly optimized to maximize the score of a reward model. When the reward model is not robust enough, it is easy to over-optimize so the prover overfits to the imperfect proxy reward. This

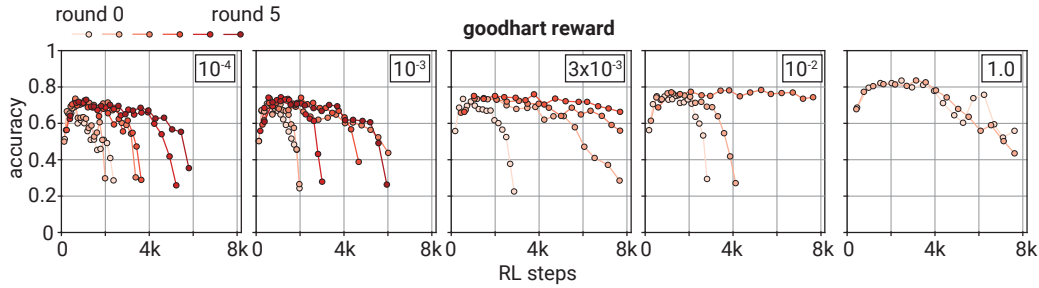


Figure 10: **Iterative goodharting prover training dynamics across verifier sizes 10^{-4} to 10^0 .** Overoptimization happens later at higher rounds, with some degradation for one verifier size (10^{-3}). Larger verifiers lead to better maximum prover accuracy and reach better robustness in fewer rounds.

phenomenon is called ‘goodharting’ or ‘reward hacking’ (Gao et al., 2023; Skalse et al., 2022). The verifier can learn from the incorrect and correct samples from the same prover.

In more details, for each round, we train the prover using PPO with reward (4) until its training accuracy drops below 0.2. Next, we sample balanced correct and incorrect samples on the verifier split of the training data for all prover checkpoints throughout PPO. The samples are then mixed into the verifier training dataset for the next round. We still keep the total number of samples per problem constant for training the verifier in each round.

Figure 10 shows the training dynamics. We early stop each round when the prover’s training accuracy tanks below 0.2 on average in the last 80 RL steps. Across verifier sizes 10^{-4} to 10^0 of the prover, the number of steps it takes for the prover to over-optimize generally increase. Up to size 10^{-2} , the larger the verifier is, the fewer number of training rounds it requires to reach perfect goodharting robustness within 8k RL steps. Verifiers of medium size ($3 \times 10^{-3} \sim 10^{-2}$) benefit most from the iterative goodharting training, where as smaller ($\leq 10^{-3}$) and larger (10^0) verifiers benefit less. We further evaluated the robustness of verifier trained using goodharting reward under attacks from the SRC reward (2) and the alternative CGC reward (3), and the verifiers’ robustness go up sometimes but not always (Figures S7a, S7b). The balanced best-of- n robustness increases (Figure S8).

However, when we showed the samples from the goodharting training to humans trainers, there is no significant difference in human judge accuracy across rounds (Fig. S6), although Fig. S6 is from a pilot study that has some design flaws (see Appendix I). Overall, our results on the iterative goodharting training show that it increases verifier soundness to a certain extent, but we do not have enough evidence showing that it improves human legibility.

6 DISCUSSION

6.1 LIMITATIONS

Only domains that allow natural language justifications. We studied the math domain because natural language explanations are common and often required for building trust in mathematical conclusions. However, in some domains it is difficult to support conclusions with natural language explanations. For example recognizing particular faces is easy for humans, but it’s hard to communicate to others in natural language why a particular face belongs to a particular person.

Need for ground truth answers and reliance on generalization. One prerequisite for our setup is being able to certify correctness of answers of solutions produced by the prover. For the grade school math domain we rely on having a training dataset of problems with known ground truth answers (Cobbe et al., 2021). We anticipate this setup to be less relevant to domains where collecting any ground truth is too hard or expensive. One possible approach in this case could be to hope for favorable out-of-distribution generalization properties of legibility from solutions to problems with known ground truth answers to other domains. We don’t study out-of-domain generalization in this paper because we train on only one dataset, but we expect that training on a diverse set of domains

could potentially lead to good generalization from domains with ground truth labels to the domains without labels (Burns et al., 2023).

Conflating the model’s chain-of-thought with legible solutions. By training the CoTs to be legible, we potentially limited the provers’ accuracy. A better method could be training a first model using CoT to maximize accuracy, and then training a second model to turn the CoT used to produce the answer into a legible justification to humans. Hence, our measurement of accuracy-legibility tradeoff may be overly pessimistic; the prover may be able to achieve legibility at less or no cost to accuracy. For the correctness-only baseline, we could have early stopped at less accurate but more legible checkpoint. However, we don’t know of a principled way to do early stopping for legibility.

Dataset diversity and difficulty. We conducted our empirical study on only one dataset (GSM). The dataset is relatively easy and the solutions are quite standardized. Using ChatGPT to augment this dataset could result in some incorrect labels, but based on preliminary data analysis we did not see significant difference between verifier test accuracies trained on synthetic vs. real data. We would like to extend the checkability training framework to harder math datasets and coding, as well as fuzzier domains such as human preference data (Ouyang et al., 2022). Our method is fully general as long as there is a finite set of final answer labels.

Initialization with human-written math derivations. Our prover initialization has relatively high performance in GSM. We used a prover that has been lightly finetuned on a variety of math data to learn the correct format, and the finetuning dataset contains some of the GSM trainset data. We believe this doesn’t qualitatively affect our results and only makes it easier for the helpful prover to produce correct answers. Figure S9 shows that initialization with a base model without math finetuning leads to qualitatively similar training dynamics.

Heterogeneous experiment set-ups. We have different stopping criteria for different training schemes presented throughout the paper. We trained the prover for a fixed number of RL steps in each round for certain training schemes (4k steps in Fig. 6a, 5c, 8k steps in Fig. 4, 5c, S4, S5), whereas in Fig. 5, 9, 10 we trained until early stopping criteria are met (the criteria are already discussed in those section). We do not observe qualitative different between early stopping and non early stopping rounds in our experiments.

Hyperparameters. We did not tune all hyperparameters thoroughly for all training schemes in this paper. The main hyperparameter we tuned are the ratios of samples from each of the previous rounds. We tuned the ratio for one verifier size (10^{-3}) for the SRC reward based on whether training robustness increases across rounds, and used the same ratio for other sizes with SRC and CGC rewards, which could result in other training schemes being worse in robustness and legibility. We separately tuned the ratio for one verifier size (10^{-3}) for the iterative goodharting training. We used the default ChatGPT post-training hyperparameters such as the optimizer, learning rate, or KL regularization coefficient for prover and verifier training.

6.2 FUTURE WORK

We studied the setting in which correctness labels are available at training time for all problems. A more realistic scenario is one where not all answers have a trusted final correctness label, in which case the verifier would rely on unsupervised signals such as consistency (Wang et al., 2023) and convincingness (Khan et al., 2024). Unsupervised methods for improving legibility are an important future direction.

Another direction is developing a method to turn an illegible solution into a legible one. The CoTs that the model first uses to solve a problem are not legible to a smaller model per se but we could train a translation model that turns the ‘scratch space’ CoT to a well-written proof. This approach might be favorable over directly training the CoT to be legible since exerting any optimization pressure on the CoT may limit prover performance as well as exacerbate deceptive alignment (i.e., the model hiding its own intent) (Denison et al., 2024; Hubinger et al., 2024).

The training schemes we consider are adaptations of single-turn simultaneous games. With an interactive tool such as ChatGPT, the verifier can interact with the prover in more rounds which